

ANALISIS ARSITEKTUR WEB SERVICE

Studi Kasus Aplikasi WisataKu

Nama : Ahmad Damanhuri

NIM : SI21240001

Prodi : Sistem Informasi

1. Analisis Studi Kasus WisataKu

WisataKu merupakan studi kasus aplikasi wisata yang dikembangkan sebagai backend API untuk platform informasi, ulasan, dan reservasi destinasi wisata. Aplikasi ini memungkinkan wisatawan mencari destinasi wisata, membaca dan menulis ulasan, serta melakukan reservasi tiket kunjungan secara daring. Sementara itu, admin memiliki fungsi untuk mengelola data destinasi.

Studi kasus WisataKu memiliki beberapa entitas bisnis utama yang digunakan sebagai model Prisma, yaitu Destinasi, Kategori, User, Ulasan, Reservasi, dan Fasilitas.

Entitas	Atribut Utama
Destinasi	id, nama, kategori, lokasi, deskripsi, hargaTiket, ratingRata
Kategori	id, namaKategori
User	id, nama, email, passwordHash, role
Ulasan	id, destinasiId, userId, rating, komentar, tanggal
Reservasi	id, userId, destinasiId, tanggalKunjungan, jumlahTiket, status, totalHarga
Fasilitas	id, destinasiId, namaFasilitas

Dengan karakteristik tersebut, WisataKu membutuhkan arsitektur Web Service yang dapat menyediakan layanan backend bagi client serta mengelola komunikasi antara aplikasi dan basis data.

Pengembangan WisataKu dilakukan secara bertahap. Modul menggunakan TypeScript, Node.js, NestJS, Prisma ORM, dan MariaDB sebagai teknologi utama dalam pembangunan backend.

2. Konsep Web Service dan API

2.1 Web Service

Web Service merupakan layanan yang menyediakan fungsi atau data melalui jaringan sehingga dapat digunakan oleh aplikasi lain. Dalam konteks WisataKu, Web Service berfungsi sebagai bagian backend yang menyediakan layanan kepada client.

Client dapat mengirimkan permintaan kepada backend WisataKu untuk memperoleh atau mengolah data yang berkaitan dengan destinasi wisata, ulasan, reservasi, dan data lainnya.

Modul praktikum membangun Web Service WisataKu menggunakan stack:

- TypeScript
- Node.js
- NestJS
- Prisma ORM
- MariaDB

Dengan demikian, Web Service menjadi penghubung antara aplikasi client dengan logika bisnis serta penyimpanan data pada backend WisataKu.

2.2 API

API atau Application Programming Interface merupakan antarmuka yang memungkinkan suatu perangkat lunak berkomunikasi dengan perangkat lunak lainnya.

Dalam WisataKu, API digunakan sebagai antarmuka antara client dengan backend. Client tidak berkomunikasi langsung dengan database, tetapi mengirimkan request kepada API yang kemudian diproses oleh backend.

Secara sederhana:

Client → API/Backend → Database

API menyediakan aturan komunikasi sehingga client mengetahui layanan yang dapat digunakan dan bagaimana berkomunikasi dengan backend.

3. HTTP dalam Web Service WisataKu

HTTP merupakan salah satu ruang lingkup utama Tugas 1. Dalam arsitektur Web Service, HTTP digunakan sebagai dasar komunikasi antara client dan layanan API.

Melalui HTTP, client dapat mengirimkan request kepada backend WisataKu dan menerima response dari backend.

Secara sederhana:

Client → HTTP Request → Backend API → HTTP Response → Client

HTTP menjadi dasar komunikasi yang kemudian digunakan dalam pengembangan REST API pada tahap berikutnya. Pada Tugas 1, pembahasan HTTP difokuskan pada perannya sebagai dasar komunikasi Web Service, bukan pada perancangan endpoint secara detail.

4. Gaya API

Dalam pengembangan sistem, terdapat beberapa gaya atau pendekatan API.

4.1 REST

REST (Representational State Transfer) merupakan gaya arsitektur API yang berorientasi pada resource.

Dalam WisataKu, resource dapat berupa:

- Destinasi
- Kategori
- User
- Ulasan
- Reservasi
- Fasilitas

REST memanfaatkan metode HTTP dan representasi data seperti JSON dalam komunikasi API.

Contohnya, resource Destinasi dapat digunakan sebagai dasar layanan API untuk mengambil, menambahkan, memperbarui, atau menghapus data destinasi.

Pada modul, REST kemudian diterapkan lebih lanjut pada perancangan endpoint dan implementasi RESTful API menggunakan NestJS, Prisma, dan MariaDB.

4.2 GraphQL

GraphQL merupakan pendekatan API yang memungkinkan client menentukan data yang ingin diminta melalui query.

Pada WisataKu, GraphQL dapat digunakan untuk memperoleh data yang diperlukan oleh client tanpa harus mengambil seluruh data yang tersedia.

Modul menempatkan implementasi GraphQL pada tahap berikutnya menggunakan `@nestjs/graphql` dan Apollo Server. Oleh karena itu, dalam Tugas 1 GraphQL dibahas sebagai salah satu gaya API, sedangkan implementasinya dilakukan pada tahap lanjutan.

4.3 SOAP

SOAP (Simple Object Access Protocol) merupakan protokol pertukaran pesan yang menggunakan format XML dan memiliki struktur komunikasi yang lebih formal.

SOAP banyak digunakan pada sistem yang membutuhkan kontrak komunikasi yang terstruktur.

Perbedaannya dengan REST adalah REST merupakan gaya arsitektur yang memanfaatkan konsep resource dan HTTP, sedangkan SOAP merupakan protokol pertukaran pesan dengan format dan aturan komunikasi yang lebih formal.

4.4 gRPC

gRPC merupakan framework komunikasi yang menggunakan Protocol Buffers dan umumnya berjalan di atas HTTP/2.

gRPC dirancang untuk komunikasi antarlayanan dengan kebutuhan komunikasi yang efisien. Dalam arsitektur sistem terdistribusi, gRPC dapat digunakan sebagai salah satu pendekatan komunikasi antar-service. Hal tersebut relevan ketika sistem berkembang menjadi arsitektur microservices.

4.5 Perbandingan Gaya API

Aspek	REST	GraphQL	SOAP	gRPC
Pendekatan	Resource-based	Query-based	Protocol/message-based	RPC
Format utama	Umumnya JSON	Query & response	XML	Protocol Buffers
Komunikasi	HTTP	HTTP	HTTP dan protokol terkait	HTTP/2
Pengambilan data	Berdasarkan resource/endpoint	Client menentukan data	Berdasarkan operasi/service	Berdasarkan method
Penggunaan	Web API umum	Data yang kompleks/fleksibel	Sistem dengan kontrak formal	Komunikasi antarlayanan
Konteks WisataKu	API utama	Tahap lanjutan	Pembanding	Relevan untuk komunikasi service

5. Arsitektur Sistem Terdistribusi

Sistem terdistribusi terdiri dari beberapa komponen yang bekerja sama melalui jaringan. Dalam konteks WisataKu, client, backend API, service, dan database merupakan bagian yang saling berkomunikasi.

Beberapa prinsip yang perlu diperhatikan adalah client-server, stateless, dan layered system.

5.1 Client-Server

Pada model client-server, client meminta layanan kepada server.

Dalam WisataKu:

- Client dapat berupa aplikasi mobile dan aplikasi web admin.
- Server berupa backend NestJS.
- Backend memproses request dan memberikan response kepada client.

Gambaran sederhananya:

Aplikasi Mobile → Backend NestJS ← Web Admin

Backend menjadi pusat layanan yang menerima request dari berbagai client.

5.2 Stateless

Prinsip stateless berarti setiap request dari client diproses secara independen dan server tidak bergantung pada penyimpanan state sesi client pada server untuk memahami request berikutnya.

Dalam API, informasi yang diperlukan untuk memproses request dikirimkan melalui request tersebut.

Penerapan prinsip stateless membantu API menjadi lebih mudah dikembangkan dan memungkinkan request diproses tanpa ketergantungan terhadap state sesi tertentu.

5.3 Layered System

Layered system membagi sistem ke dalam beberapa lapisan berdasarkan tanggung jawabnya.

Dalam backend WisataKu, pembagian tersebut dapat digambarkan secara konseptual:

Client → API/Controller → Service/Business Logic → Prisma ORM → MariaDB

Setiap bagian memiliki tanggung jawab berbeda.

- Client mengirimkan request.
- Controller menangani request dan routing.
- Service menangani logika bisnis.
- Prisma ORM menangani akses data.
- MariaDB menyimpan data.

Pembagian ini membantu menjaga struktur aplikasi agar lebih terorganisasi.

6. Analisis Arsitektur WisataKu

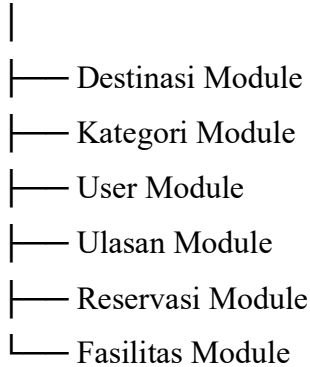
6.1 Arsitektur Monolitik

Arsitektur monolitik menempatkan komponen aplikasi dalam satu aplikasi/backend yang dideploy sebagai satu kesatuan.

Untuk tahap awal WisataKu, pendekatan monolitik dapat dibuat secara modular.

Contohnya:

WisataKu Backend



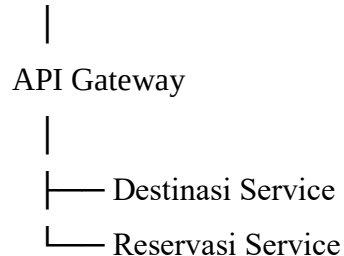
Walaupun berada dalam satu aplikasi, setiap bagian tetap memiliki batas modul yang jelas.

7. Arsitektur Microservices

Pada arsitektur microservices, aplikasi dibagi menjadi beberapa service yang dapat dikembangkan dan dideploy secara terpisah.

Contoh pembagian yang sesuai dengan konteks WisataKu:

Client



Masing-masing service menangani tanggung jawab bisnis tertentu.

Pendekatan ini dapat digunakan ketika sistem sudah berkembang dan kebutuhan pemisahan service menjadi lebih jelas.

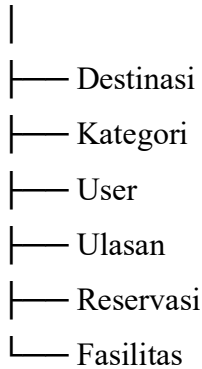
8. Analisis Monolitik Modular vs Microservices

Untuk tahap awal pengembangan WisataKu, modul mengarahkan penggunaan arsitektur monolitik modular, bukan langsung memecah sistem menjadi microservices.

Alasannya adalah pengembangan dilakukan secara bertahap dan sistem masih berada pada tahap pembangunan awal. Dengan monolitik modular, seluruh fungsi backend masih berada dalam satu aplikasi tetapi dapat dipisahkan berdasarkan modul.

Contohnya:

WisataKu



Pendekatan tersebut membuat pengembangan awal lebih sederhana sekaligus tetap memberikan batas modul yang jelas.

NestJS mendukung struktur modular melalui Module, Controller, dan Provider/Service. Struktur tersebut juga memungkinkan batas modul sudah jelas sejak awal sehingga sistem dapat dikembangkan lebih lanjut apabila pada tahap berikutnya membutuhkan pemisahan menjadi microservices.

9. Alasan Tidak Langsung Menggunakan Microservices

Microservices tidak langsung digunakan pada tahap awal karena WisataKu masih dibangun secara bertahap sebagai satu backend.

Jika sistem langsung dipecah menjadi banyak service, pengembangan akan melibatkan komunikasi antar-service dan pengelolaan beberapa komponen secara terpisah.

Sebaliknya, monolitik modular memungkinkan pengembang:

1. membangun satu backend terlebih dahulu;
2. memisahkan fungsi berdasarkan modul;
3. menjaga struktur kode tetap terorganisasi;
4. mengembangkan fitur secara bertahap;
5. memiliki batas modul yang jelas untuk kemungkinan pengembangan berikutnya.

Dengan demikian, penggunaan monolitik modular pada tahap awal merupakan bagian dari strategi pengembangan bertahap yang dijelaskan dalam modul.

10. NestJS sebagai Backend WisataKu

NestJS merupakan framework Node.js berbasis TypeScript yang digunakan sebagai framework backend pada modul.

NestJS menyediakan struktur modular dengan komponen utama seperti:

- Module
- Controller
- Provider/Service

Pada WisataKu, Controller digunakan untuk menangani request HTTP, sedangkan Service digunakan untuk logika bisnis.

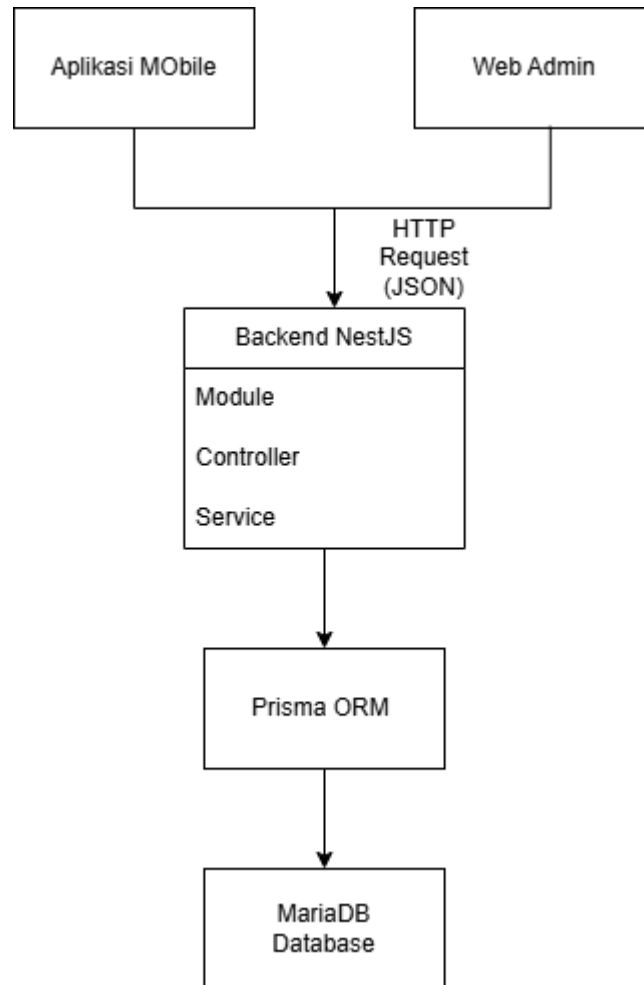
Contoh struktur:

src/

```
|— main.ts
|— app.module.ts
|
└─ destinasi/
    |— destinasi.module.ts
    |— destinasi.controller.ts
    └─ destinasi.service.ts
```

11. Analisis Alur Arsitektur WisataKu

Secara konseptual, alur komunikasi WisataKu dapat digambarkan:



Pada diagram tersebut, aplikasi mobile dan web admin berkomunikasi dengan backend NestJS. Backend kemudian menangani proses aplikasi dan mengakses database melalui Prisma ORM.

12. Kesimpulan

Berdasarkan analisis studi kasus WisataKu, Web Service berfungsi sebagai layanan backend yang menjadi penghubung antara client dengan sistem aplikasi dan basis data.

HTTP menjadi dasar komunikasi antara client dengan backend API. REST digunakan sebagai pendekatan utama dalam pengembangan API WisataKu, sedangkan GraphQL dibahas sebagai pendekatan API yang akan diimplementasikan pada tahap berikutnya. SOAP dan gRPC dapat digunakan sebagai pembanding terhadap pendekatan komunikasi API lainnya.

Dari sisi arsitektur, WisataKu menggunakan pendekatan monolitik modular pada tahap awal. Pendekatan ini memungkinkan backend dikembangkan dalam satu aplikasi dengan pembagian modul yang jelas seperti Destinasi, Kategori, User, Ulasan, Reservasi, dan Fasilitas.

Penggunaan NestJS mendukung pendekatan tersebut karena menyediakan struktur modular melalui Module, Controller, dan Provider/Service. Dengan batas modul yang jelas, sistem tetap dapat dikembangkan lebih lanjut apabila kebutuhan untuk menggunakan microservices muncul pada tahap berikutnya.